

Revision 06-08-09 • Benchmark Statistics, Streaming & the GPU Dashboard

Interactive revision of [lesson 06](#), [lesson 08](#) and [lesson 09](#): each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 • Tail latency in a 20-call agent MISSED

Covers: 06

QUESTION

One agent task makes 20 sequential LLM calls. The API's SLO is $P99 = 2$ s (1% of calls take longer). What fraction of tasks hit at least one slow call, and how should per-call targets be set for agents?

How to solve it: flip to "nothing goes wrong"

1. One call is fast with probability 0.99.
2. All 20 fast: $0.99^{20} \approx \mathbf{0.82}$.
3. At least one slow: $1 - 0.82 \approx \mathbf{18\% \text{ of tasks}}$. Shortcut for small q : $k \times q = 20 \times 1\% \approx 20\%$.
4. **Per-call target**: to keep slow tasks to 1%, each call may be slow only $\sim 1\% \div 20 = 0.05\%$ of the time, i.e. a **P99.95** per-call target. A per-call P99 behaves like a task-level P82.

Computing 0.99^{20} by hand

- Rough: $(1 - x)^n \approx 1 - n \cdot x = 1 - 0.2 = 0.80$.
- Better: $(1 - x)^n \approx e^{(-n \cdot x)} = e^{(-0.2)} \approx 1 - 0.2 + 0.02 = \mathbf{0.82}$ (using $e^y \approx 1 + y + y^2/2$).
- Exact: square repeatedly: $0.99^2 = 0.9801 \rightarrow ^4 \approx 0.9606 \rightarrow ^8 \approx 0.9227 \rightarrow ^{16} \approx 0.8515$; then $^{16} \times ^4 \approx 0.8515 \times 0.9606 \approx 0.818$.

RE-DRILL

For "at least one" questions, compute the chance of "none" and subtract from 1. Remember $(1 - x)^n \approx e^{(-n \cdot x)}$: 50 calls at P99 $\rightarrow e^{(-0.5)} \approx 0.61 \rightarrow \sim 39\%$ of tasks see a slow call.

Q2 • Is a 3.5% speedup real? MISSED

Covers: 06

QUESTION

5 runs before average 20.0 ms TPOT; 5 runs after average 19.3 ms; run-to-run $\sigma \approx 0.8$ ms. "3.5% faster, ship it." Is it real?

How to think about it: wobble, not formulas

1. **One run wobbles** by $\sigma \approx 0.8$ ms around the true value.
2. **Averaging calms it, slowly**: an average of n runs wobbles $\sigma \div \sqrt{n} = 0.8 \div \sqrt{5} \approx 0.8 \div 2.2 \approx \mathbf{0.36}$ ms (the standard error).
3. **A difference of two averages wobbles more**: both are uncertain, and two independent wobbles combine as $\sqrt{2} \approx 1.4 \times: 0.36 \times 1.4 \approx \mathbf{0.5}$ ms.
4. **"Pretty sure" $\approx$ 2 wobbles** ($\sim 95\%$ confidence): noise band $\approx 2 \times 0.5 \approx \mathbf{\pm 1.0}$ ms.
5. **Compare**: observed difference = $20.0 - 19.3 = \mathbf{0.7}$ ms, inside ± 1.0 ms.
Could be luck: not proven, don't ship yet.
6. **To settle it**: $4 \times$ the runs (20 each) halves the band to ± 0.5 ms; if the 0.7 ms gap holds, it's real.

RE-DRILL

Re-derive it as a story: one run $\rightarrow$ average of $n \rightarrow$ difference of two $\rightarrow \times 2$ for confidence.

Q3 · A benchmark claiming 2,000 tokens/s **PARTIAL**

Covers: 08, 01

QUESTION

A script times `model.generate()` for Llama 3 8B, batch 1, one H100, and reports 2,000 tokens/s. Why must it be wrong, and what are two likely bugs?

Answer

- **Impossible**: batch-1 decode is capped by **memory bandwidth**: $16 \text{ GB} \div 3.35 \text{ TB/s} \approx 4.8 \text{ ms}$ per token $\rightarrow \leq \sim 200$ tokens/s. 2,000 is $10 \times$ over.
- **Bug 1, no GPU sync**: CUDA is asynchronous; Python queues work and returns. Stopping the timer without `torch.cuda.synchronize()` (or CUDA events) times the queuing, not the GPU, so the rate looks far too high.
- **Bug 2, counting the prompt**: dividing prompt + generated tokens by the time instead of generated tokens only ($1.5 \times$ too high in lesson 08's example).
- **Fix**: synchronize before reading the clock, count only generated tokens, and report TTFT and TPOT separately.

RE-DRILL

Sanity-check every benchmark against the floor before believing it.

Q4 · "GPU-util is 100%, we need more GPUs" **PARTIAL**

Covers: 09

QUESTION

One user chats with Llama 3 8B on an H100. Dashboard: GPU-util 100%, power 350 W of a 700 W cap, decode 6 ms per token. Is the GPU maxed out? Is half the power cap a problem? How do you measure real usage?

Answer

- **GPU-util is a duty cycle:** the fraction of time any kernel ran. One kernel on 1 of 132 SMs reads 100%. It says nothing about how busy the GPU is.
- **Half power is normal:** memory-bound decode leaves the math units waiting, so the board draws less. Real power problems show as power-cap or thermal throttling in clock event reasons.
- **Measure MBU** = achieved bandwidth ÷ peak bandwidth: $16 \text{ GB} \div 6 \text{ ms} \approx 2.67 \text{ TB/s}$, $\div 3.35 \text{ TB/s} \approx 80\%$. MFU = $(15 \text{ GFLOP} \div 6 \text{ ms}) \div 989 \text{ TFLOPS} \approx 0.25\%$.
- **Verdict:** not maxed out. One user at ~80% bandwidth with idle compute, so batching more users is nearly free. Don't buy GPUs.

RE-DRILL

MBU = bytes per step ÷ step time ÷ peak bandwidth; MFU = FLOPs per step ÷ step time ÷ peak FLOPS.

Exercise · See benchmark noise by simulation SOLID

Covers: 06

QUESTION

Simulate long-tailed latencies. (A) Estimate P99 from $n = 100, 1,000, 10,000$ samples, 200 times each, and measure how much the estimates wobble. (B) Brute-force the 20-call agent: what fraction of tasks hit a slow call?

```
def p99_spread(n, repeats=200):
    estimates = [p99([latency() for _ in range(n)]) for _ in range(repeats)] # 200 benchmarks
    mean = statistics.mean(estimates)
    rel_spread = statistics.stdev(estimates) / mean
    return mean, rel_spread

def slow_task_fraction(calls=20, q=0.01, tasks=100_000):
    slow_tasks = 0
    for _ in range(tasks):
        if any(random.random() < q for _ in range(calls)): # slow if ANY call is slow
            slow_tasks += 1
    return slow_tasks / tasks
```

Samples per benchmark	P99 estimate	Wobble
100	2.97 s	±16%
1,000	3.17 s	±6%
10,000	3.20 s	±2%

Slow tasks: **18.3%**, matching $1 - 0.99^{20} \approx 18\%$.

What it shows: - Wobble shrinks like $\sqrt{n}$: $100\times$ the samples $\rightarrow \sim 8\times$ less wobble (the $\sigma/\sqrt{n}$ from Q2). - Small samples **under-report the tail**: at $n = 100$ only ~ 1 sample lands beyond P99, so the estimate averages 2.97 s vs the true ~ 3.20 s. A 100-request benchmark flatters your tail.